Logo ENSIT · Logo du ministère de l'Éducation (optionnel)

École Nouvelle Supérieure d'Ingénieurs et de Technologie (ENSIT)

Entreprise d'accueil : SYNELIA

Mémoire de fin d'études

En vue de l'obtention du diplôme d'Ingénieur en Sciences Informatiques

Option : Génie Logiciel

Thème

Étude, conception et mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée : cas de l'entreprise SYNELIA

Présenté par

[Nom patronymique] [Prénom]

Encadrement

Pédagogique : [Titre Nom Prénom]

Entreprise : [Nom Prénom]

[Fonction du maître de stage]

Année académique : 2023 - 2024

École Nouvelle Supérieure d'Ingénieurs et de Technologie (ENSIT)

Entreprise d'accueil : SYNELIA

Page de garde

Titre du mémoire

Étude, conception et mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée : cas de l'entreprise SYNELIA

Présenté par : [Nom patronymique] [Prénom]

Encadreur pédagogique : [Titre Nom Prénom]

Maître de stage : [Nom Prénom]

[Fonction du maître de stage]

diplôme d'Ingénieur en Sciences Informatiques

Option : Génie Logiciel

Année académique : 2023 - 2024

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadreur pédagogique, **[Titre Nom Prénom]**, pour son accompagnement, sa disponibilité et la qualité de ses conseils tout au long de ce travail.

Mes sincères remerciements s'adressent également à mon maître de stage, **[Nom Prénom]**, pour sa confiance, son encadrement au sein de SYNELIA et la richesse des échanges techniques et méthodologiques qui ont nourri ce projet.

Je remercie l'ensemble de l'équipe SYNELIA pour son accueil, sa bienveillance et son implication dans le suivi du projet SyneliaQuizApp.

Mes remerciements vont aussi au corps enseignant et administratif de l'ENSIT, ainsi qu'aux membres du jury qui consacreront du temps à l'évaluation de ce mémoire.

Enfin, j'adresse une pensée reconnaissante à ma famille et à mes proches pour leur soutien constant durant mon parcours académique.

Résumé

À finaliser en dernier

Le résumé devra tenir sur 1 à 1,5 page maximum et synthétiser :

- le contexte du projet
- la problématique traitée
- les objectifs poursuivis
- la méthodologie adoptée
- les technologies mobilisées
- les résultats obtenus
- les mots-clés

[Insérer ici le résumé final du mémoire en français.]

Abstract

To be finalized last

The abstract should remain concise and aligned with the final French summary.

[Insert here the final English abstract.]

Sommaire

Introduction 1
1. Présentation générale du projet 5
2. Étude de l'existant et état de l'art 11
3. Analyse des besoins 16
4. Conception de la solution 22
5. Environnement technique et outils 28
6. Implémentation de la solution 33
7. Tests et validation 39
8. Présentation des résultats 45
9. Limites et perspectives 48
Conclusion 50
Références bibliographiques 51
Annexes 52

Avant-propos

L'avant-propos présente le cadre institutionnel du mémoire. Il situe le travail au regard de la formation d'ingénieur en sciences informatiques dispensée à l'ENSIT et du contexte professionnel offert par l'entreprise d'accueil SYNELIA.

Contenu attendu

- présentation de l'ENSIT
- présentation de la formation et du diplôme
- présentation de SYNELIA
- cadre du stage de fin d'études

[Rédiger ici un avant-propos institutionnel, sans entrer directement dans l'analyse technique du projet.]

Glossaire

API	Application Programming Interface
Angular	Framework frontend utilisé pour l'interface web monopage
ATS	Applicant Tracking System
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
JPA	Java Persistence API
JWT	JSON Web Token
MySQL	Système de gestion de base de données relationnelle
REST	Style d'architecture des services web
RGPD	Règlement Général sur la Protection des Données
SPA	Single Page Application
Spring Boot	Framework backend Java retenu pour le projet
Typst	Système de composition utilisé pour le mémoire
UML	Unified Modeling Language
WebSocket	Protocole de communication bidirectionnelle en temps réel

À ENRICHIR

Ajouter ici les autres sigles réellement utilisés dans le mémoire au fur et à mesure de la rédaction.

Introduction

À l'ère du digital, la gestion du recrutement connaît une transformation profonde. Les entreprises ne peuvent plus se contenter d'un traitement fragmenté des candidatures, de validations manuelles dispersées et d'outils hétérogènes pour piloter l'ensemble du parcours candidat.

Dans ce contexte, le projet **SyneliaQuizApp** a été conçu pour répondre à un besoin concret de l'entreprise SYNELIA : disposer d'une plateforme centralisée de gestion des campagnes de recrutement, de suivi des candidatures, d'évaluation technique et de coordination des intervenants internes.

Éléments obligatoires à développer

- présentation du sujet
- problématique
- justification du choix de la thématique
- définition des concepts clés
- méthodologie générale adoptée
- annonce du plan
- phrase de clôture ouvrant sur les perspectives

Le recrutement occupe aujourd'hui une place stratégique dans la compétitivité des entreprises de services numériques. Dans un environnement où les profils techniques sont fortement sollicités, la capacité à identifier rapidement les bons candidats, à évaluer leurs compétences avec cohérence et à maintenir une communication fluide tout au long du processus devient un avantage organisationnel décisif. Or, dans de nombreuses structures, ces opérations restent encore dispersées entre plusieurs outils, plusieurs interlocuteurs et plusieurs supports de suivi difficilement consolidables.

C'est dans ce contexte que s'inscrit le présent mémoire, consacré à l'étude, à la conception et à la mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée pour l'entreprise SYNELIA. Le sujet ne porte pas uniquement sur la réalisation d'un logiciel, mais sur la transformation d'un processus métier complet : centraliser les candidatures, structurer les campagnes de recrutement, intégrer un quiz technique, coordonner les évaluations, suivre les décisions et améliorer l'expérience vécue par les candidats comme par les recruteurs.

La problématique à laquelle répond ce travail peut être formulée ainsi : *comment concevoir une plateforme unifiée capable de digitaliser le recrutement, de standardiser*

l'évaluation technique et de renforcer la traçabilité des décisions, tout en restant adaptée aux contraintes opérationnelles d'une entreprise en croissance ? Derrière cette question se trouvent plusieurs enjeux concrets : éviter la dispersion des informations, réduire la charge manuelle, harmoniser les critères d'analyse, sécuriser les accès et rendre le pilotage du recrutement plus lisible.

Le choix de cette thématique se justifie à la fois par sa pertinence pour SYNELIA et par son intérêt du point de vue de l'ingénierie logicielle. Pour l'entreprise, il s'agit de répondre à un besoin réel de structuration des recrutements et de maîtrise des échanges internes et externes. Pour l'étudiant ingénieur, le projet constitue un terrain particulièrement riche, car il mobilise l'analyse des besoins, la modélisation métier, la conception d'une architecture web complète, l'intégration de mécanismes de sécurité, la gestion des données, ainsi que la validation d'un produit à forte densité fonctionnelle.

Plusieurs concepts clés traversent ce mémoire. La **campagne de recrutement** désigne le cadre organisationnel dans lequel sont publiés les besoins et suivis les candidats. Le **parcours candidat** correspond à la succession des interactions publiques, depuis la soumission jusqu'au suivi de l'avancement. Le **quiz technique** constitue un mécanisme de pré-évaluation automatisée destiné à objectiver une partie de l'analyse des compétences. Enfin, la **traçabilité** renvoie à la capacité du système à conserver les états, les décisions, les permissions et les événements utiles à la compréhension du processus.

La méthodologie adoptée dans ce travail repose sur une progression en quatre temps. Il a d'abord fallu analyser le besoin métier à partir du cahier des charges, des notes fonctionnelles et des flux réellement couverts par l'application. Une phase de conception a ensuite permis de structurer l'architecture, les responsabilités des modules, les règles de gestion et les principaux objets manipulés par le système. La réalisation a été menée de manière incrémentale sur un backend Spring Boot, un frontend Angular et une base MySQL, avec des validations régulières. Enfin, un travail de recul critique a permis d'évaluer les résultats obtenus, les limites rencontrées et les perspectives d'évolution du produit.

Le mémoire est organisé en quatre parties complémentaires. La première partie pose le cadre théorique et le contexte du projet, en présentant l'entreprise, la problématique et l'état de l'existant. La deuxième partie est consacrée à l'analyse des besoins et à la conception de la solution. La troisième partie décrit la réalisation effective de l'application, son environnement technique et la démarche de validation retenue. La quatrième partie dresse le bilan des résultats obtenus, puis ouvre sur les limites du travail réalisé et les perspectives d'amélioration envisageables.

Ainsi, au-delà de la seule production d'une application web, ce mémoire vise à montrer comment une démarche d'ingénierie logicielle peut contribuer à transformer

un processus métier sensible en un système structuré, évolutif et porteur de valeur pour l’entreprise.

Première partie

Cadre théorique et contexte du projet

Fondements du besoin, positionnement du sujet et justification de la solution

CHAPITRE 1

Présentation générale du projet

Ce premier chapitre a pour rôle de cadrer le projet dans son environnement métier, organisationnel et académique. Il introduit les besoins de SYNELIA, les objectifs poursuivis et le périmètre retenu pour la solution développée.

Sources prioritaires

- z_mémoire/CAHIER_DES_CHARGES_FINAL.md
- z_mémoire/template_memoire/index.html
- cartographies fonctionnelles et notes métier

1.1 Contexte du projet

Le projet SyneliaQuizApp est né d'un besoin métier concret exprimé par SYNELIA dans un contexte de croissance et de diversification des recrutements. En tant qu'entreprise spécialisée dans les services informatiques et le conseil en technologies, SYNELIA doit identifier, évaluer et intégrer des profils variés allant de jeunes diplômés à des consultants ou experts plus expérimentés. Cette dynamique de recrutement rend rapidement visibles les limites d'un traitement fondé sur des supports dispersés, des échanges manuels et des pratiques peu standardisées.

Avant la mise en place d'une solution dédiée, le processus de recrutement reposait sur une combinaison d'emails, de fichiers de suivi, de formulaires externes, d'échanges informels et de validations éclatées entre plusieurs acteurs. Une telle organisation peut fonctionner à faible volume, mais elle devient fragile dès que le nombre de candidatures augmente, que plusieurs recruteurs doivent collaborer sur les mêmes dossiers ou que l'entreprise souhaite objectiver davantage ses décisions.

La mise en œuvre d'une plateforme interne de recrutement répond donc à un double objectif. D'une part, il s'agit d'optimiser le traitement opérationnel des candidatures en centralisant les données, les états et les interactions. D'autre part, il s'agit de doter l'entreprise d'un système capable de produire un processus plus lisible, plus équitable et plus traçable, tant pour les recruteurs que pour les candidats.

1.1.1 L'entreprise SYNELIA et ses besoins

SYNELIA évolue dans le domaine des services numériques et du conseil en technologies. Ce positionnement implique une forte dépendance à la qualité des ressources humaines, en particulier pour les profils techniques capables d'intervenir sur des projets variés. Dans ce contexte, le recrutement ne constitue pas une activité ponctuelle, mais une fonction support essentielle au développement de l'entreprise.

Le cahier des charges mentionne plusieurs dizaines de recrutements par an, avec des profils couvrant différents niveaux d'expérience et plusieurs spécialités. Cette fréquence crée un besoin de structuration durable : l'entreprise ne peut pas dépendre d'une logique artisanale à chaque nouveau recrutement. Elle a besoin d'un environnement numérique capable d'assurer la continuité entre la diffusion d'une campagne, la réception des candidatures, l'évaluation des compétences, la coordination interne et la décision finale.

1.1.2 Le processus de recrutement traditionnel

Dans un fonctionnement traditionnel, le processus de recrutement suit une chaîne relativement connue. Une offre ou une campagne est d'abord diffusée par différents canaux, tels que l'email, les réseaux professionnels ou les plateformes d'emploi. Les candidatures sont ensuite reçues, triées et relues manuellement selon des critères qui peuvent varier d'un recruteur à l'autre.

Vient ensuite une phase d'évaluation, souvent réalisée à partir d'entretiens techniques, de questionnaires externes ou de jugements informels portés sur le dossier du candidat. Les entretiens doivent être planifiés, confirmés et suivis, ce qui multiplie les échanges. Enfin, une décision de présélection, de poursuite ou de rejet est prise, puis communiquée au candidat avec plus ou moins de formalisation.

Pris isolément, chacun de ces maillons paraît simple. Mais lorsqu'ils sont répartis entre plusieurs outils et plusieurs personnes, ils génèrent rapidement des pertes d'information, des lenteurs et une forte dépendance à la mémoire individuelle des intervenants.

1.1.3 Les défis et limitations actuels

Les principales difficultés du processus initial tiennent d'abord à la fragmentation des outils. Les candidatures peuvent arriver par plusieurs canaux, les informations sont réparties entre boîtes mail, documents partagés et fichiers de suivi, et l'historique complet d'un candidat devient difficile à reconstituer. Cette dispersion nuit directement à la réactivité et à la qualité du pilotage.

Une seconde limite concerne l'absence de standardisation des évaluations. Sans grille commune ni mécanisme homogène de test technique, la comparaison entre candidats devient moins objective et dépend fortement de l'interprétation de chaque recruteur. Ce problème est particulièrement sensible dans un environnement où les compétences techniques doivent être évaluées avec rigueur.

Enfin, le manque de traçabilité fragilise la gouvernance du recrutement. Il devient difficile de savoir pourquoi une décision a été prise, quelle source de candidature est la plus efficace, quel acteur est intervenu à quel moment ou quels retards affectent l'expérience candidat. La lourdeur organisationnelle qui en découle peut impacter à la fois la qualité du recrutement et l'image de l'entreprise.

1.1.4 La nécessité d'une solution digitale

Dans ce contexte, une plateforme interne unifiée apparaît comme une réponse pertinente pour plusieurs raisons. Elle permet d'abord de centraliser les informations et d'offrir un référentiel commun à tous les acteurs du recrutement. Elle permet ensuite d'automatiser certaines opérations répétitives, comme la vérification d'email, l'envoi de notifications, la mise à disposition du quiz ou la consolidation de certains états métier.

Une telle solution favorise également la standardisation du processus. En structurant les campagnes, les statuts, les évaluations et les permissions, elle réduit la variabilité des pratiques et facilite la comparaison entre candidatures. Enfin, elle améliore la transparence du parcours candidat en donnant accès à un suivi plus clair, ce qui contribue à renforcer la qualité de l'expérience utilisateur et, par extension, l'image employeur de SYNELIA.

1.2 Objectifs du projet

Les objectifs du projet doivent être compris à deux niveaux complémentaires. D'un côté, il existe des objectifs stratégiques liés à la transformation du processus de recrutement et à l'amélioration de la performance organisationnelle. De l'autre, le projet poursuit des objectifs opérationnels précis, directement traduits dans les modules et parcours de SyneliaQuizApp.

1.2.1 Objectifs généraux

L'objectif général principal consiste à **digitaliser le processus de recrutement** en remplaçant une logique éclatée par une plateforme web centralisée. Cette digitalisation vise à réduire les manipulations manuelles, à structurer les échanges et à rendre plus visibles les différentes étapes du parcours candidat.

Le deuxième objectif général est **d'automatiser l'évaluation technique** grâce à l'intégration d'un système de quiz configurable, permettant de pré-filtrer ou de préqualifier certains profils de manière plus homogène. Cette automatisation n'a pas vocation à remplacer totalement le jugement humain, mais à l'appuyer par un socle de vérification plus objectif.

Enfin, le projet cherche à **améliorer la collaboration interne** entre les acteurs du recrutement. Cela suppose de partager les informations utiles, de distribuer les droits de

manière adaptée, de suivre les interventions de chacun et de permettre une meilleure coordination des évaluations, des entretiens et des décisions.

1.2.2 Objectifs spécifiques

Sur le plan opérationnel, SyneliaQuizApp vise à couvrir plusieurs besoins concrets. Il s'agit d'abord de permettre la création de campagnes de recrutement configurables, associées à des formulaires et à des droits d'accès adaptés. L'application doit ensuite offrir un parcours candidat cohérent, depuis la soumission initiale jusqu'au suivi public de la candidature.

Le projet doit également prendre en charge le quiz technique, sa diffusion, sa reprise éventuelle, sa correction automatique ou manuelle et sa restitution dans le contexte global du recrutement. À cela s'ajoutent la planification des entretiens, la collecte d'évaluations collaboratives, la production de décisions, la gestion des notifications et l'administration des données de référence nécessaires au fonctionnement du système.

En pratique, ces objectifs spécifiques traduisent une ambition claire : couvrir le cœur du cycle de recrutement dans une même plateforme, avec une profondeur fonctionnelle suffisante pour soutenir un usage réel.

1.3 Périmètre fonctionnel

Le périmètre de la solution a été défini de manière à couvrir le cœur du besoin exprimé par SYNELIA, tout en conservant une limite raisonnable pour une première version cohérente. L'application se concentre donc sur la gestion du recrutement, de la candidature à la décision, avec les modules nécessaires à la circulation de l'information et à l'évaluation des profils.

1.3.1 Modules couverts par l'application

Les modules effectivement couverts par l'application comprennent l'authentification et la gestion des accès, la gestion des campagnes de recrutement, les formulaires et questionnaires, le dépôt et le suivi des candidatures, le quiz technique, la planification des entretiens, les évaluations collaboratives, les décisions, les notifications et les tableaux de bord d'administration ou de suivi.

À ces modules s'ajoutent plusieurs briques transverses indispensables, comme la gestion des modèles d'email, la configuration SMTP, les référentiels métier, les permissions par rôle et par entité, ainsi que les vues techniques d'administration permettant d'exploiter l'application dans la durée.

1.3.2 Acteurs du système

Le système met en interaction quatre grandes catégories d'acteurs. Le **super administrateur** intervient sur les paramètres sensibles et les ressources globales de l'application.

L'**administrateur** gère les données de référence, certains écrans techniques et une partie des configurations nécessaires au fonctionnement de la plateforme.

Le **recruteur** ou gestionnaire métier constitue l'acteur principal du processus opérationnel : il crée les campagnes, suit les candidatures, planifie les entretiens, consulte les évaluations et participe aux décisions. Le **candidat**, quant à lui, interagit avec l'application à travers des parcours publics sécurisés, principalement pour soumettre sa candidature, passer un quiz, consulter son suivi ou répondre à des sollicitations liées à son dossier.

1.3.3 Frontières du projet

Le projet inclut donc le cœur du processus de recrutement, mais il n'a pas vocation à couvrir l'ensemble du système d'information RH de l'entreprise. Des fonctionnalités comme l'intégration directe avec les job boards, la visioconférence intégrée, la gestion de la paie ou des contrats, l'application mobile native, le multilingue ou le matching automatique par intelligence artificielle sont volontairement laissées hors du périmètre de cette version.

Cette délimitation est importante d'un point de vue méthodologique. Elle permet de concentrer l'effort sur une version robuste du processus principal, plutôt que de disperser le développement sur des fonctions annexes encore prématurées. Le choix retenu privilégie donc la cohérence métier, la stabilité de l'architecture et la qualité du noyau fonctionnel.

1.4 Contraintes du projet

Le projet s'inscrit dans un ensemble de contraintes qu'il convient de reconnaître explicitement. Sur le plan technique, l'application doit rester accessible depuis un navigateur web moderne, proposer des temps de réponse acceptables sur les opérations courantes et conserver une architecture suffisamment claire pour être maintenue et étendue. Elle doit également composer avec plusieurs technologies complémentaires et avec un modèle de données évolutif.

Sur le plan organisationnel, la solution doit pouvoir être prise en main par des profils différents, ce qui implique un effort de lisibilité des écrans, une gestion des droits compréhensible et un accompagnement au changement. Le recrutement étant un processus transverse, le système ne peut être utile que s'il parvient à articuler les besoins de plusieurs acteurs sans les noyer dans une complexité excessive.

Des contraintes réglementaires et opérationnelles pèsent également sur le projet, en particulier autour de la protection des données personnelles des candidats, de la traçabilité des décisions et de la maîtrise des communications sortantes. Enfin, comme tout projet mené dans le cadre d'un stage, SyneliaQuizApp a dû être développé dans un temps

limité, avec une nécessité permanente de prioriser entre profondeur fonctionnelle, qualité technique et faisabilité dans le délai imparti.

CHAPITRE 2

Étude de l'existant et état de l'art

Ce chapitre positionne la solution développée par rapport aux pratiques existantes et aux solutions du marché. Il sert également à justifier les choix technologiques retenus pour la mise en œuvre du système.

> **Important pour la V1**
>
> Ce chapitre peut être rédigé sans citations dans un premier temps. La bibliographie et les références académiques seront injectées dans une phase ultérieure.

2.1 Étude de l'existant

L'existant chez SYNELIA se caractérisait par une organisation fonctionnelle identifiable, mais fortement dispersée sur le plan outillé. Les candidatures pouvaient être collectées par différents canaux, puis suivies à l'aide d'emails, de fichiers bureautiques, de tableaux de suivi et de formulaires externes. Cette organisation avait l'avantage d'être rapide à mettre en place au départ, sans nécessiter de solution métier spécifique. Elle permettait aussi une certaine souplesse locale dans la manière de traiter les dossiers.

Toutefois, cette souplesse avait pour contrepartie un manque de centralisation. Les informations utiles à l'analyse d'une candidature n'étaient pas toujours réunies au même endroit, les critères d'évaluation variaient selon les acteurs, et la reconstitution du parcours complet d'un candidat devenait coûteuse en temps. L'existant présentait donc une valeur opérationnelle minimale, mais peinait à soutenir une logique de croissance, de collaboration structurée et de traçabilité fiable.

Il convient néanmoins de souligner que cet existant mettait déjà en évidence les besoins réels de l'entreprise. Les recruteurs avaient déjà identifié la nécessité de diffuser des campagnes, de recueillir des informations standardisées, de tester les compétences, de suivre les statuts et de produire des décisions. En ce sens, le projet SyneliaQuizApp ne part pas d'un vide fonctionnel ; il cherche plutôt à rationaliser et à intégrer des pratiques déjà présentes, mais encore insuffisamment outillées.

2.2 Benchmark des solutions existantes

Le marché propose aujourd'hui de nombreuses solutions de recrutement, parmi lesquelles des ATS généralistes comme Workable ou Greenhouse, ainsi que des plateformes plus spécialisées dans l'évaluation technique comme HackerRank, Codility ou TestGorilla. Les premières se distinguent généralement par leur capacité à structurer les campagnes, centraliser les candidats, suivre les statuts et produire des tableaux de bord. Les secondes mettent davantage l'accent sur la conception de tests, la notation automatisée et l'évaluation comparée des compétences.

Ces solutions offrent des fonctionnalités avancées et une forte maturité produit, mais elles ne répondent pas toujours de manière optimale à un besoin interne spécifique. D'une part, elles peuvent imposer des modèles de workflow peu flexibles par rapport à l'organisation réelle de l'entreprise. D'autre part, elles introduisent souvent une dépendance à une plateforme externe, à ses coûts, à ses conventions et à ses limites de personnalisation. Enfin, lorsqu'il faut articuler très finement les permissions, les parcours publics, les états métier et les règles propres à l'entreprise, une solution générique peut s'avérer moins adaptée qu'un outil pensé sur mesure.

Le benchmark met donc en évidence un arbitrage classique. Les solutions du marché permettent de démarrer rapidement avec un haut niveau de finition. En revanche, une solution interne comme SyneliaQuizApp peut offrir une meilleure maîtrise du modèle de données, des parcours utilisateurs, des règles métier et de l'évolution future du produit, à condition de supporter l'effort initial de conception et de développement.

2.3 État de l'art technologique

L'état de l'art retenu pour ce projet repose sur une architecture web en couches, devenue aujourd'hui un standard pour les applications métier. Le backend est porté par un framework serveur capable d'exposer des services structurés, d'assurer la sécurité, de gérer la persistance et d'orchestrer les règles métier. Le frontend, quant à lui, s'appuie sur un framework orienté composants permettant de construire des interfaces riches, interactives et organisées autour de parcours utilisateurs distincts.

Dans ce paysage, **Spring Boot** s'impose comme une référence pour les applications Java orientées API. Il facilite la mise en place de services REST, l'intégration avec la sécurité, la persistance via JPA et l'organisation modulaire du code. **Angular** représente de son côté une solution robuste pour les applications monopage complexes, en particulier lorsqu'il faut structurer de nombreux écrans, partager des composants et maintenir une logique de navigation cohérente entre plusieurs profils d'utilisateurs.

La couche de données relationnelle reste également un choix pertinent pour un domaine aussi structuré que le recrutement. Une base comme **MySQL** permet de modéliser proprement les relations entre campagnes, candidatures, formulaires, tentatives de quiz, entretiens, évaluations, décisions et permissions. Enfin, l'architecture **REST** fournit un cadre clair pour les échanges entre frontend et backend, tandis que les principes modernes de sécurité web imposent une séparation rigoureuse entre authentification, autorisation, validation des entrées, contrôle des accès publics et protection des données sensibles.

2.4 Justification des choix technologiques

Le choix de Spring Boot, Angular et MySQL se justifie d'abord par leur complémentarité. Spring Boot permet de structurer un backend robuste, modulaire et bien adapté à la complexité métier du projet. Angular apporte, côté interface, un cadre solide pour gérer une application riche en écrans, en rôles utilisateurs et en interactions conditionnées par les permissions. MySQL, enfin, répond efficacement aux besoins de cohérence relationnelle et d'intégrité du modèle de données.

Ces technologies ont également été retenues pour leur maturité et pour la qualité de leur écosystème. Elles disposent d'outils, de bibliothèques, de conventions et de ressources suffisantes pour accompagner un projet de taille intermédiaire sans imposer une complexité disproportionnée. Dans le cadre d'un projet de fin d'études, cet équilibre est important : il faut pouvoir produire une solution techniquement sérieuse tout en restant dans un environnement de développement maîtrisable.

Enfin, ces choix servent directement les objectifs du projet. La séparation frontend/backend rend le système plus évolutif. L'usage d'une base relationnelle soutient la richesse des relations métier. Les frameworks retenus facilitent la mise en œuvre de la sécurité, des notifications, de la documentation d'API et de la maintenance progressive du produit. Le choix technologique n'est donc pas seulement un choix d'outils ; il participe pleinement à la qualité architecturale de la solution proposée.

2.5 Critique de l'existant et solution proposée

L'analyse de l'existant et du marché montre clairement que le problème de SYNELIA n'est pas l'absence totale d'outils, mais l'absence d'un outil unificateur, cohérent avec ses besoins réels. Les pratiques historiques permettaient certes d'assurer un traitement minimal des candidatures, mais au prix d'une forte charge manuelle, d'une faible visibilité globale et d'une standardisation insuffisante. À l'inverse, les solutions du marché offrent une réponse industrialisée, mais pas toujours parfaitement alignée sur les contraintes, les préférences et le niveau de contrôle attendu par l'entreprise.

SyneliaQuizApp se positionne précisément dans cet espace. La solution proposée vise à centraliser les campagnes, les candidatures, les quiz, les entretiens, les évaluations et les décisions dans une plateforme unique, tout en intégrant des mécanismes de permissions fines, de suivi public candidat et de configuration transverse. Sa valeur ajoutée tient donc moins à la reproduction de fonctions déjà connues qu'à leur articulation dans un système unifié, pensé pour le contexte spécifique de SYNELIA.

Ce chapitre permet ainsi de justifier le projet à un double niveau : fonctionnellement, il répond à de réelles insuffisances du processus existant ; technologiquement, il s'appuie sur des choix cohérents avec l'état de l'art tout en assumant une logique de personnalisation et de maîtrise interne. La suite du mémoire montrera comment cette orientation a été traduite en besoins structurés, puis en solution concrète.

Deuxième partie

Analyse et conception

Formalisation des besoins, règles de gestion et architecture de la solution

CHAPITRE 3

Analyse des besoins

Ce chapitre a pour objectif de démontrer que la solution répond à des besoins clairement identifiés, priorisés et structurés selon une logique d'ingénierie logicielle.

Sources prioritaires

- cartographies fonctionnelles
- z_notes/
- to-do/
- backlog métier et notes de debug significatives

3.1 Méthodologie d'analyse

L'analyse des besoins a été menée selon une démarche progressive, ancrée dans les usages réels du projet plutôt que dans une modélisation abstraite déconnectée de l'implémentation. Le point de départ a été le cahier des charges fonctionnel, enrichi ensuite par les cartographies métier, les notes de debug, les listes de tâches et les observations issues des validations successives du produit. Cette approche a permis de confronter en permanence le besoin théorique, le comportement du code et les arbitrages effectivement retenus pendant le développement.

Plusieurs parties prenantes ont implicitement structuré cette analyse. Le candidat représente l'utilisateur externe principal, avec des attentes de simplicité, de clarté et de continuité dans son parcours. Les recruteurs et évaluateurs incarnent les besoins métiers centraux liés au pilotage des campagnes, à la lecture des dossiers, à l'évaluation et à la décision. Les administrateurs, enfin, portent les exigences de configuration, de gouvernance et de maintien du système dans la durée.

Méthodologiquement, les besoins ont été regroupés en modules fonctionnels, puis relus à travers des critères transverses : sécurité, permissions, cohérence des états, traçabilité, qualité de l'expérience utilisateur et capacité d'évolution. Ce choix de lecture par parcours et par règles métier est particulièrement important dans un projet comme SyneliaQuizApp, où une fonctionnalité ne peut pas être correctement comprise si elle est isolée de son contexte de campagne, de permission ou de pipeline.

3.2 Besoins fonctionnels

Les besoins fonctionnels du projet peuvent être structurés en plusieurs blocs applicatifs complémentaires, chacun correspondant à un segment du processus de recrutement. Cette structuration permet de relier les attentes métier aux modules effectivement conçus dans la solution et de montrer que le produit répond à une logique de parcours complet plutôt qu'à une accumulation de fonctionnalités indépendantes.

3.2.1 Authentification et gestion des utilisateurs

Le système doit permettre l'authentification des utilisateurs internes, la gestion de leurs rôles et le contrôle précis de leurs droits d'accès. Les administrateurs et recruteurs ont besoin d'accéder à des espaces différenciés, avec des permissions cohérentes selon leur responsabilité réelle. L'application doit donc gérer à la fois des droits globaux liés au rôle système et des droits contextualisés, portés par une campagne ou par une entité particulière.

Pour les parcours publics, le besoin est différent : le candidat ne doit pas disposer d'un accès interne classique, mais d'un ensemble de points d'entrée sécurisés, restreints à sa propre candidature. Cela implique des mécanismes dédiés de vérification d'email, de codes de suivi et de liens contextualisés, distincts du modèle d'authentification des utilisateurs internes.

3.2.2 Gestion des données de référence

Le projet nécessite un socle de données de référence administrables, sans lequel le recrutement ne peut pas être structuré durablement. Cela inclut notamment les départements, les spécialités, les questions de quiz, les réponses attendues, les formulaires et plusieurs référentiels transverses nécessaires au fonctionnement métier et aux écrans d'administration.

Le besoin ne porte pas seulement sur la création de ces données, mais aussi sur leur cohérence, leur réutilisation et leur évolution. Une question peut être partagée par plusieurs formulaires, un formulaire peut être versionné, et certaines données doivent être historisées indirectement pour éviter que des modifications ultérieures ne déforment la lecture des campagnes déjà utilisées. L'administration des référentiels constitue donc une base essentielle de la qualité du système.

3.2.3 Campagnes de recrutement

La campagne de recrutement constitue l'unité de pilotage centrale du système. Le besoin métier impose de pouvoir créer une campagne, définir son état de cycle de vie, lui associer un pipeline, lui rattacher un ou plusieurs formulaires, générer des liens de diffusion et partager son accès avec des collaborateurs internes selon des permissions fines.

La campagne doit également servir de point de convergence pour le suivi des candidatures, des entretiens, des évaluations et des décisions. Elle doit donc supporter à la fois une logique de configuration et une logique de lecture consolidée. L'entreprise attend en particulier une meilleure visibilité sur les campagnes actives, sur les profils qu'elles visent et sur les actions qu'elles autorisent aux différents intervenants.

3.2.4 Candidatures et suivi candidat

Le système doit offrir un parcours de candidature public simple et fiable, permettant au candidat de renseigner ses informations, de transmettre ses pièces, de vérifier son adresse email et d'obtenir un moyen de suivi autonome de son dossier. Ce besoin est fondamental, car il conditionne à la fois la qualité de la donnée collectée et la perception du sérieux de l'entreprise par les candidats.

Au-delà de la soumission initiale, la solution doit permettre un suivi public contextualisé : consultation de l'état d'avancement, visibilité sur les actions possibles, reprise éventuelle de certaines étapes, réponse à une convocation et, lorsque la règle métier l'autorise, retrait de la candidature ou demande de correction d'information. Le système doit donc gérer une frontière claire entre les actions publiques autorisées et les opérations strictement réservées aux équipes internes.

3.2.5 Quiz technique

Le quiz technique répond à un besoin de standardisation et d'objectivation de l'évaluation initiale des candidats. L'application doit pouvoir rattacher un formulaire publié à une campagne, générer une tentative de quiz pour un candidat donné, gérer le temps imparti si nécessaire, sauvegarder les réponses et calculer un score exploitable dans la suite du pipeline.

Dans la cible fonctionnelle retenue, ce besoin va plus loin qu'un simple test ponctuel. Le système doit également pouvoir gérer des occurrences explicites de quiz, distinguer les tentatives successives, autoriser certaines reprises dans des conditions contrôlées et articuler les résultats du quiz avec l'avancement global de la candidature. Cette capacité d'occurrence est importante pour préparer des pipelines plus riches et pour améliorer la traçabilité du parcours candidat.

3.2.6 Entretiens, évaluations et décisions

Le projet doit permettre la planification d'entretiens reliés à une étape précise du pipeline, la communication de la convocation au candidat, la prise en compte de sa réponse et la mise à jour du statut correspondant. L'entretien n'est donc pas conçu comme un simple rendez-vous, mais comme un événement métier ayant des effets sur l'état du dossier, sur la disponibilité des actions et sur la suite du processus de recrutement.

L'évaluation doit être collaborative, détaillée et contextualisée. Plusieurs évaluateurs peuvent être amenés à produire des observations ou des notes à propos d'un même can-

didat, avec des critères plus ou moins pondérés. Enfin, le système doit permettre la prise de décision, qu'il s'agisse d'une pré-sélection, d'une décision finale, d'une annulation ou d'un retour arrière sur une occurrence donnée du pipeline. Le besoin central ici est la cohérence entre jugement métier, historisation et évolution du dossier candidat.

3.2.7 Notifications et dashboard

Le besoin ne se limite pas à stocker des données ; il consiste aussi à faire circuler l'information au bon moment. L'application doit donc générer des notifications internes et des emails pour signaler les événements significatifs du processus : vérification, convocation, réponse candidat, résultat d'une demande, décision ou défaut de configuration critique.

Parallèlement, le dashboard doit fournir des vues synthétiques permettant de suivre l'activité, de filtrer les dossiers, d'identifier les actions en attente et de faciliter le pilotage quotidien par les équipes. Le système doit ainsi rendre visibles les éléments réellement utiles à la décision, sans noyer l'utilisateur dans une masse d'informations non contextualisées.

3.3 Besoins non fonctionnels

Les besoins non fonctionnels jouent un rôle majeur dans la réussite du projet. En matière de **performance**, l'application doit offrir des temps de réponse compatibles avec un usage fluide sur les opérations courantes, en particulier sur les listes de candidatures, les écrans de détail et les parcours publics. La **sécurité** exige une séparation stricte entre surface publique et surface interne, un contrôle rigoureux des permissions, une validation des entrées et une protection adaptée des données personnelles et des secrets techniques.

L'**ergonomie** constitue un autre besoin important, car le système s'adresse à des profils différents et doit rester compréhensible malgré la richesse du domaine métier. La **maintenabilité** impose une architecture claire, un découpage modulaire, des conventions stables d'API et une capacité à faire évoluer les règles métier sans dégrader l'ensemble. La **disponibilité** et la **fiabilité** exigent enfin que les composants essentiels restent exploitables dans le cadre d'un usage régulier, avec des mécanismes de build et de validation reproductibles.

La **conformité** complète cet ensemble. Dans un domaine manipulant des données de candidature, l'application doit respecter des exigences de protection des données, de conservation raisonnée, de traçabilité et de justification des décisions. Ces besoins non fonctionnels expliquent une grande partie des choix d'architecture et de modélisation présentés dans le chapitre suivant.

3.4 Diagrammes des cas d'utilisation

Les cas d'utilisation globaux du système peuvent être organisés par acteur principal. Pour le candidat, les usages clés sont : accéder à une campagne, soumettre une candidature, vérifier son email, passer un quiz, consulter son suivi, répondre à une convocation et solliciter certaines corrections ou actions publiques autorisées. Pour le recruteur, les cas d'utilisation majeurs sont : créer et configurer une campagne, associer des formulaires, consulter les candidatures, corriger un test si nécessaire, planifier des entretiens, saisir des évaluations et prendre des décisions.

Pour l'administrateur et le super administrateur, les cas d'utilisation se concentrent sur la gestion des référentiels, des utilisateurs, des rôles, des permissions, des templates d'emails, de la configuration SMTP, de la configuration applicative et des écrans techniques de contrôle. Ces acteurs assurent la disponibilité et la gouvernance du système plus qu'ils ne participent directement au recrutement opérationnel.

Les diagrammes associés, à insérer dans la version finale du mémoire, pourront être présentés selon deux niveaux : un diagramme global montrant les grandes interactions entre acteurs et système, puis des diagrammes ciblés par profil pour détailler les flux candidat, recruteur et administration. Cette organisation permettra de conserver une vue synthétique sans perdre la lisibilité des cas d'usage spécialisés.

3.5 Règles de gestion

Les règles de gestion les plus structurantes du projet peuvent être résumées selon plusieurs axes. D'abord, la campagne suit un cycle de vie contrôlé, avec des transitions autorisées entre brouillon, attente, ouverture, pause et clôture. Une campagne ouverte doit disposer d'un pipeline valide, et l'accès public dépend de règles précises comme l'état de la campagne, la validité du lien source, la fenêtre de dates et les limites de candidature.

Ensuite, la candidature suit une logique pilotée par le pipeline. Le `application_progress` constitue la source de vérité de l'avancement, tandis que certains champs visibles au niveau de la candidature jouent un rôle de miroir ou de synthèse. Les étapes peuvent être validées, échouées, parfois ignorées selon la configuration, et certaines actions comme le quiz, l'entretien ou la décision sont rattachées à des occurrences explicites du pipeline afin de préserver la cohérence métier.

Les permissions constituent un troisième pilier de règles. Une action n'est pas autorisée uniquement parce qu'un utilisateur possède un rôle global donné ; elle peut aussi dépendre d'un droit contextualisé sur une campagne ou une entité spécifique. Cette règle est particulièrement importante pour la correction des tests, la planification des entretiens, le suivi d'une campagne ou la saisie d'une évaluation.

Enfin, plusieurs règles de validation garantissent la fiabilité du système : vérification d'email avant certains accès publics, contrôle des données requises avant envoi d'email, gestion des statuts de correction de quiz, justification obligatoire pour certaines décisions critiques, et restrictions sur les mises à jour partielles afin de préserver l'intégrité des objets métier. L'ensemble de ces règles montre que la solution repose sur une modélisation orientée gouvernance autant que sur une logique de simple saisie.

CHAPITRE 4

Conception de la solution

Ce chapitre traduit les besoins en architecture, structures de données, flux applicatifs et conventions d'implémentation.

> **Point de rédaction**
>
> Rédiger ce chapitre comme si la cible multi-occurrences était totalement finalisée, conformément au cadrage retenu pour le mémoire.

4.1 Architecture globale du système

La solution adopte une architecture web en couches, fondée sur une séparation nette entre l'interface utilisateur, les services métier et la persistance. Le frontend Angular constitue la couche de présentation. Il prend en charge les pages publiques candidat, les tableaux de bord internes, les composants partagés, les services d'appel API et la logique d'interaction propre à chaque profil utilisateur. Le backend Spring Boot joue le rôle de couche applicative centrale : il expose les endpoints, applique les règles métier, contrôle les permissions, orchestre les transitions de pipeline et dialogue avec la base de données.

La persistance repose sur une base relationnelle MySQL enrichie par des migrations Flyway, ce qui permet de faire évoluer le schéma de manière versionnée. Autour de ce noyau, des services auxiliaires comme Redis, MinIO ou les composants d'envoi d'emails complètent l'architecture physique lorsque le scénario métier l'exige. L'ensemble forme une architecture distribuée simple, mais extensible, adaptée à une application métier de taille intermédiaire.

Sur le plan logique, le backend est lui-même organisé en sous-couches lisibles : exposition web, services métier, accès aux données, sécurité, configuration et utilitaires. Cette organisation limite le couplage direct entre l'interface et la persistance. Elle permet surtout de centraliser les règles métier dans des services spécialisés, ce qui est indispensable dans un domaine où les transitions d'état, les permissions, les agrégats et les effets de bord doivent rester cohérents dans tout le système.

4.2 Conception de la base de données

La conception de la base de données suit une logique relationnelle riche, cohérente avec la nature fortement structurée du domaine. Au niveau conceptuel, les entités principales s'organisent autour de quelques noyaux : utilisateurs et permissions, campagnes et pipelines, formulaires et questions, candidatures et parcours candidat, entretiens et évaluations, décisions et notifications. Chaque noyau possède ses propres relations, mais l'ensemble converge vers la candidature comme objet pivot du recrutement.

Dans la cible multi-occurrences, la modélisation du pipeline joue un rôle déterminant. Une candidature n'est plus simplement associée à un statut global, mais à une série d'occurrences portées par `application_progress`. Les quiz sont représentés par des tentatives explicites, les entretiens sont rattachés à une occurrence de pipeline, et les décisions elles-mêmes peuvent être ancrées sur une étape précise. Cette orientation permet de gérer proprement des répétitions, des retours arrière, des annulations et des historiques détaillés sans perdre la cohérence d'ensemble.

Au niveau logique puis physique, les principales entités manipulées sont notamment : `user`, `campaign`, `pipeline_template`, `pipeline_stage`, `form`, `form_version`, `question`, `application`, `application_progress`, `application_quiz_attempt`, `candidate_answer`, `interview`, `candidate_interview`, `evaluation`, `decision`, `notification` et les tables de permissions ou de configuration. Ce modèle relationnel soutient à la fois la traçabilité fine du processus et la production de vues synthétiques nécessaires au pilotage.

4.3 Diagrammes de classes

Les diagrammes de classes du backend doivent mettre en évidence les responsabilités principales de l'application plutôt qu'une simple hiérarchie technique. La classe de démarrage de l'application ouvre naturellement l'ensemble, mais les éléments centraux de la modélisation résident surtout dans les services métier et les entités principales. Des classes comme celles qui gèrent les candidatures, les campagnes, les entretiens, les évaluations, les décisions ou les permissions incarnent les règles de gestion les plus structurantes.

Du côté métier, les services associés à la candidature, au pipeline, aux entretiens, aux évaluations, aux décisions, aux permissions et aux templates d'email doivent apparaître comme des orchestrateurs de cas d'usage. Ils coordonnent les repositories, appliquent les validations, résolvent les droits et consolident les agrégats nécessaires au frontend. Les entités JPA, quant à elles, représentent les objets persistés du système et matérialisent les relations essentielles entre campagnes, candidats, étapes de pipeline, quiz, entretiens et décisions.

Les diagrammes pourront donc être structurés en plusieurs vues : une vue métier centrée sur les entités du recrutement, une vue applicative montrant les principales classes de service, puis éventuellement une vue plus ciblée sur les objets transverses comme les permissions, les notifications et la configuration. Cette approche rendra la conception plus lisible qu'un diagramme unique trop dense.

4.4 Diagrammes de séquence

Plusieurs séquences clés peuvent être retenues pour illustrer le comportement dynamique du système. La première concerne la création d'une candidature : le candidat accède à une campagne publique, soumet ses informations, le backend valide l'éligibilité de la campagne, crée la candidature, génère un code de suivi, puis déclenche l'envoi d'un email de vérification ou de confirmation selon le contexte. Cette séquence met en évidence l'articulation entre accès public, création métier, persistance et communication sortante.

La deuxième séquence concerne le quiz technique. Après vérification des prérequis, le système résout la bonne occurrence de quiz, fournit les questions autorisées, enregistre progressivement les réponses et calcule le résultat au moment opportun. Dans la cible multi-occurrences, cette séquence doit aussi montrer le rattachement des réponses à une tentative donnée, afin d'éviter toute ambiguïté entre score courant, historique et synthèse visible dans les écrans d'administration.

Une troisième séquence importante porte sur l'entretien et la décision. Le recruteur planifie un entretien sur une occurrence de pipeline, le candidat reçoit une convocation et peut répondre, les évaluateurs produisent ensuite leurs appréciations, puis une décision est saisie et renvoyée dans le flux d'avancement. Cette séquence illustre particulièrement bien la logique d'orchestration métier du système, dans laquelle plusieurs acteurs et plusieurs objets évoluent de manière coordonnée.

4.5 Diagrammes d'activités et d'états

Les diagrammes d'activités et d'états doivent représenter la candidature comme un processus évolutif plutôt que comme un simple enregistrement doté d'un champ de statut. L'activité globale peut être lue comme une progression entre dépôt, vérification, quiz, pré-sélection, entretien, évaluation et décision, avec des branches possibles de réussite, d'échec, d'annulation, de report ou de retrait selon les cas métier.

La modélisation par états gagne en précision dans la cible retenue, car chaque étape importante du parcours possède sa propre occurrence et son propre état d'avancement. Une candidature peut ainsi avoir un état global de synthèse tout en conservant, dans son historique, plusieurs états d'étapes : en attente, en cours, réussi, échoué, ignoré, archivé,

fermé ou réouvert selon la nature de l'opération. Cette conception renforce la capacité du système à expliquer les transitions réellement survenues.

Les diagrammes correspondants devront donc articuler deux niveaux : un niveau macro, orienté lecture métier du dossier candidat, et un niveau micro, orienté pipeline et occurrences. Cette double lecture permettra d'expliquer à la fois le comportement visible du système et les mécanismes internes qui garantissent sa cohérence.

4.6 Maquettes et interfaces

Les maquettes et interfaces de la solution s'organisent autour de trois univers d'usage distincts. Le premier est l'espace public candidat, qui comprend les écrans de candidature, de vérification d'email, de quiz et de suivi. L'enjeu principal de cette interface est la simplicité : le candidat doit comprendre rapidement ce qu'on attend de lui, où il en est et quelles actions lui sont encore accessibles.

Le deuxième univers est celui du recruteur. Il comprend les listes de campagnes, les vues de candidatures, les écrans de planification d'entretien, les interfaces d'évaluation, les espaces de décision et les tableaux de bord. Ici, la logique d'interaction doit favoriser la lecture rapide, le filtrage, l'accès au contexte utile et la sécurisation des actions sensibles par des droits explicites ou par des confirmations adaptées.

Le troisième univers est celui de l'administration. Il regroupe les référentiels, les paramètres globaux, les templates d'emails, la configuration SMTP, les permissions et certaines vues techniques du système. Les maquettes associées devront mettre en évidence une exigence particulière de lisibilité et de cohérence, car elles supportent des opérations de gouvernance dont dépend une grande partie du comportement applicatif.

4.7 Conception de l'API REST

La conception de l'API repose sur une logique de services orientés métier. Les endpoints ne se limitent pas à un CRUD canonique ; ils exposent aussi des actions applicatives explicites, telles que la progression dans le pipeline, la réponse à un entretien, la publication d'un formulaire ou la création d'une occurrence contextuelle. Cette orientation rend l'API plus proche du langage métier du système et facilite le pilotage des cas d'usage complexes.

Les échanges sont essentiellement réalisés au format JSON, via des DTO dédiés servant de contrats entre frontend et backend. Les structures de requête permettent de transmettre soit des recherches par critères, soit des objets métier à créer ou à mettre à jour, parfois regroupés dans des tableaux de données. Les réponses ont pour objectif de renvoyer des objets directement exploitables par l'interface, y compris lorsqu'elles portent des champs de synthèse ou des informations dérivées utiles à l'affichage.

L'API suit également des conventions de mise à jour partielles importantes pour la stabilité du projet : certaines valeurs omises ne doivent pas être écrasées, certaines valeurs vides doivent être ignorées, et certains champs calculés ou historisés restent strictement propriétaires du backend. La gestion des erreurs doit enfin préserver un équilibre entre rigueur technique et lisibilité fonctionnelle : contrôle des entrées, refus explicite des incohérences critiques, messages exploitables pour le frontend et distinction claire entre erreur métier, erreur de validation et défaut d'infrastructure.

Troisième partie

Réalisation et mise en œuvre

Technologies utilisées, organisation du code et stratégie d'exécution du projet

CHAPITRE 5

Environnement technique et outils

Ce chapitre présente l'environnement réel dans lequel SyneliaQuizApp a été développé, testé et exécuté. Il constitue un point d'entrée concret et factuel pour la lecture technique du mémoire.

Sources prioritaires

- quizapp/pom.xml
- quizapp_front/package.json
- quizapp_front/angular.json
- docker-compose.yml

5.1 Environnement de développement

Le développement de SyneliaQuizApp a été réalisé dans un environnement de travail orienté poste client, adapté à une production incrémentale et à des validations fréquentes. Les itérations les plus récentes du projet ont été menées sous **Windows**, avec **Visual Studio Code** comme environnement principal d'édition, de navigation dans le code et de pilotage des validations. Ce choix s'explique par la complémentarité des extensions utilisées pour Java, Angular, PowerShell, Markdown et, dans le cadre de la préparation du mémoire, Typst.

L'organisation du travail s'appuie sur une séparation explicite entre le backend, le frontend, la documentation fonctionnelle et les notes de pilotage. Le dépôt contient ainsi le projet Spring Boot dans `quizapp/`, l'application Angular dans `quizapp_front/`, des scripts utilitaires, un historique de notes techniques, ainsi que les documents de cadrage et de mémoire. Cette structuration a favorisé une progression par slices fonctionnelles, chaque évolution étant accompagnée d'un support de cartographie, de validation et de capitalisation.

La gestion de version repose sur **Git**, tandis que l'exécution locale s'appuie sur des commandes simples et reproductibles. Côté backend, la présence de `mvnw` et `mvnw.cmd` permet de standardiser les builds Maven. Côté frontend, les scripts déclarés dans `package.json` centralisent le démarrage, la compilation, le mode watch et les tests. Des fichiers de configuration comme `proxy.conf.js` et `proxy.conf.json` assurent en outre

une communication transparente entre l'interface Angular et l'API locale pendant les phases de développement.

5.2 Technologies backend

Le backend de l'application repose sur **Java 21** et sur **Spring Boot 3.5.6**. Le fichier `quizapp/pom.xml` confirme ce positionnement technique à travers la propriété `java.version` fixée à 21 et l'utilisation du parent Spring Boot correspondant. Le projet est packagé au format **WAR**, ce qui permet de conserver une compatibilité de déploiement avec un serveur applicatif tout en gardant la souplesse d'un démarrage local maîtrisé.

La couche web est construite avec **Spring Web**, qui sert de socle à l'exposition des endpoints REST. À cela s'ajoutent **Spring Security** pour l'authentification et l'autorisation, **Spring WebSocket** pour les fonctionnalités temps réel, ainsi que **Spring Data JPA** pour la persistance relationnelle. La classe principale `QuizappApplication` s'appuie également sur `@EnableScheduling`, ce qui montre que l'application intègre des traitements planifiés dans son cycle d'exécution.

La persistance métier s'appuie sur **MySQL** via le connecteur `mysql-connector-j`, tandis que **Flyway** est utilisé pour gérer les migrations de schéma de manière versionnée. Ce choix est particulièrement important dans un projet dont le modèle de données a fortement évolué, notamment autour de la candidature, des occurrences de pipeline, des quiz, des entretiens et des notifications.

Plusieurs bibliothèques complémentaires renforcent la qualité de l'implémentation. **MapStruct** réduit le coût de transformation entre entités et objets d'échange, **Lombok** diminue le code répétitif, **Thymeleaf** est utilisé pour les modèles d'emails, et **Spring Context Support** avec **Jakarta Mail** assurent la gestion des envois. Le backend intègre également des briques spécialisées telles que **Spring LDAP Core**, **Apache POI** pour la manipulation de documents bureautiques, **MinIO** pour la gestion objet, **Redis** pour certains usages de support, et **Springdoc OpenAPI** pour documenter l'API.

Enfin, l'architecture serveur reste cohérente avec un projet d'ingénierie orienté maintenabilité. Le serveur Tomcat est déclaré avec le scope `provided`, la pile de sécurité est centralisée, et les règles métier sont isolées dans des classes spécialisées de la couche business. Cette organisation facilite la lisibilité du code, l'écriture de tests ciblés et l'évolution progressive du produit.

5.3 Technologies frontend

L'interface utilisateur repose sur **Angular 20.2**, comme l'indiquent les dépendances `@angular/core`, `@angular/router`, `@angular/forms` et les autres modules clés du fichier `quizapp_front/package.json`. Le projet est compilé avec **TypeScript 5.9.2**, ce qui

permet de bénéficier d'un typage strict et d'une meilleure sécurisation des contrats de données entre l'interface et le backend.

Le fonctionnement réactif de l'application s'appuie sur **RxJS 7.8**, tandis que **Zone.js** reste présent pour l'intégration au modèle de détection de changements d'Angular. Le build frontend est piloté par le builder `@angular/build:application`, configuré dans `angular.json`. Les scripts `start`, `build`, `watch` et `test` permettent respectivement de lancer le serveur de développement, de produire un build final, de travailler en compilation continue et d'exécuter les tests unitaires.

L'interface est stylée en **SCSS**, choix confirmé à la fois dans les schémas Angular et dans la configuration du projet. Cette approche facilite la factorisation des styles, l'expression de variations visuelles et la structuration des pages. Le frontend adopte par ailleurs une organisation par écrans métier, visible dans le dossier `src/app/pages`, avec des modules tels que `apply-position`, `dashboard`, `quiz`, `track-application` ou encore `verify-email`. Ce découpage favorise une lecture directe du produit par parcours utilisateur.

Le projet frontend intègre également une couche de composants mutualisés, un espace partagé, des services applicatifs, des directives, des pipes et des éléments de configuration. Cette structuration est cohérente avec une application monopage de taille intermédiaire à forte densité fonctionnelle, dans laquelle la réutilisation d'éléments d'interface et la cohérence des interactions constituent des enjeux centraux.

5.4 Base de données

La base de données principale de SyneliaQuizApp est une base **MySQL 8**, choisie pour sa robustesse, sa large diffusion en contexte web et son adéquation avec un modèle relationnel riche. Le domaine métier du recrutement manipule en effet des objets fortement liés entre eux : campagnes, formulaires, candidatures, étapes de pipeline, tentatives de quiz, entretiens, évaluations, décisions, notifications et permissions. Un SGBDR relationnel s'impose donc naturellement pour garantir l'intégrité des relations et la cohérence des transactions.

Le couplage entre le code Java et la base repose sur **JPA**, avec une organisation en entités et repositories. La présence du dossier `quizapp/src/main/java/tech/synelia/quizapp/dao/entity` d'un côté et du dossier `repository` de l'autre montre clairement cette séparation entre modélisation des tables et accès aux données. Cette approche permet d'exprimer des règles métier complexes tout en gardant un point d'entrée structuré vers la couche de persistance.

L'écosystème local n'est pas limité à MySQL. Le fichier `docker-compose.yml` montre que des services complémentaires comme **Redis 7** et **MinIO** sont également prévus dans l'environnement de développement. Redis peut servir de support aux besoins de cache ou de coordination légère, tandis que MinIO apporte une solution compatible stockage objet pour les pièces et contenus associés à l'application. Cette composition technique reflète une architecture pragmatique : un cœur transactionnel relationnel, enrichi par des services spécialisés lorsque cela apporte une valeur concrète.

5.5 Outils complémentaires

Plusieurs outils complémentaires participent à la qualité et à l'exploitabilité du projet. Côté backend, **Maven** reste l'outil de référence pour la gestion du cycle de vie, des dépendances et des tests Java. L'utilisation du wrapper Maven évite de dépendre d'une installation globale homogène sur tous les postes. Côté frontend, **npm** pilote l'installation des paquets et l'exécution des scripts Angular.

Les validations API s'appuient sur **Postman**, comme le montre le document `quizapp/POSTMAN_API_GUIDE.md`. Cet outil joue un rôle important dans un projet à forte densité d'endpoints, car il permet de rejouer rapidement des scénarios métier, de contrôler les contrats d'entrée et de sortie et de vérifier le comportement des routes publiques et privées. La présence de cartographies fonctionnelles, de notes de debug et de guides de validation montre d'ailleurs que les outils de test manuel ont été intégrés à la démarche d'ingénierie et non utilisés de manière ponctuelle.

Enfin, **Docker Compose** simplifie la mise à disposition des services auxiliaires, tandis que **Git** structure l'historique de travail et la traçabilité des évolutions. Dans le cadre de la rédaction du mémoire, l'introduction de **Typst** complète cet environnement en fournissant une chaîne de composition adaptée à un document long, structuré et versionnable.

Couche	Technologies principales	Rôle dans SyneliaQuizApp
Backend	Java 21, Spring Boot 3.5.6	Exposition de l'API REST, sécurité, logique métier et orchestration des traitements du recrutement.
Frontend	Angular 20.2, TypeScript 5.9.2	Construction de l'interface monopage, gestion des parcours candidat et recruteur, communication avec l'API.

Données	MySQL 8, Flyway	Persistance relationnelle, évolution versionnée du schéma et maintien de la cohérence métier.
Services auxiliaires	Redis 7, MinIO, Docker Compose	Support aux besoins transverses comme le cache, le stockage objet et l'environnement d'exécution local.
Validation	Maven, npm, Postman, Typst	Build, vérifications ciblées, validation des scénarios métier et production du mémoire.

CHAPITRE 6

Implémentation de la solution

Ce chapitre détaille la manière dont la solution a été concrètement construite, tant du côté backend que du côté frontend.

6.1 Organisation du code source

Le workspace du projet est organisé autour d'une séparation nette entre les composants applicatifs, la documentation fonctionnelle et les artefacts de pilotage. Le dossier `quizapp/` contient le backend Spring Boot, tandis que `quizapp_front/` regroupe l'application Angular. Autour de ces deux noyaux se trouvent des éléments de support comme `docker-compose.yml`, des scripts utilitaires, des notes de travail, des cartographies fonctionnelles, des check-lists et, désormais, l'ossature du mémoire lui-même.

Le backend adopte une organisation classique mais efficace pour un projet de gestion métier. Le point d'entrée applicatif est la classe `QuizappApplication`, et les responsabilités sont ensuite réparties entre plusieurs packages : `business` pour la logique métier, `dao` pour la persistance, `rest` pour l'exposition web, `security` pour la sécurité, `config` pour la configuration transverse, `utils` pour les utilitaires, et `websocket` pour les mécanismes temps réel. Cette architecture favorise un découplage clair entre exposition, orchestration métier et accès aux données.

Le frontend suit une logique équivalente de séparation des responsabilités. Le dossier `src/app` contient notamment des sous-espaces dédiés aux pages, aux composants réutilisables, au noyau applicatif, aux services et aux éléments partagés. Le dossier `pages` reflète directement les grands parcours métiers de l'application, alors que `shared` concentre les briques génériques utilisées dans plusieurs écrans, comme les panneaux de détail, les sélecteurs de filtre ou les composants de saisie plus spécialisés.

6.2 Implémentation du backend

L'implémentation backend repose sur une logique en couches. Les contrôleurs REST reçoivent les requêtes et délèguent le traitement aux classes de la couche `business`, lesquelles centralisent les règles applicatives. Cette couche métier constitue le cœur réel de l'application : elle orchestre les transitions d'état, les validations fonctionnelles, les

notifications, les interactions entre objets métier et les politiques de sécurité appliquées aux différentes opérations.

La couche de persistance est structurée autour des entités JPA et des repositories. Le package `dao` contient à la fois la modélisation des objets persistés et les interfaces d'accès aux données. Cette organisation permet de faire évoluer les règles métier sans mélanger les considérations de persistance avec la logique d'orchestration. Elle est particulièrement utile dans un contexte où les modèles de candidature, de quiz et d'entretien ont connu des évolutions importantes.

La configuration transverse est isolée dans des espaces dédiés, notamment pour la sécurité, les intercepteurs REST, les WebSockets et le comportement applicatif global. Cette structuration favorise la maintenabilité et rend plus explicite le rôle de chaque couche dans le système.

6.3 Implémentation du frontend

L'implémentation frontend s'inscrit dans le modèle d'une application monopage orientée parcours utilisateur. Angular est utilisé pour structurer l'interface en pages, composants et services, avec une séparation nette entre les écrans métier et les éléments réutilisables. Les pages couvrent les grands usages de la plateforme : candidature, quiz, suivi public, authentification, tableau de bord et administration métier.

Les composants partagés du dossier `shared` constituent un socle important de cohérence visuelle et fonctionnelle. Ils évitent de redévelopper les mêmes interactions dans plusieurs modules et facilitent l'homogénéité de l'interface. Les services applicatifs portent quant à eux les appels HTTP et les logiques transverses nécessaires au dialogue avec le backend, tandis que le noyau applicatif héberge les mécanismes plus globaux comme les guards, la configuration et certains comportements d'infrastructure côté client.

Cette architecture frontend présente un avantage majeur dans un projet comme SyneliaQuizApp : elle permet de faire évoluer une fonctionnalité locale sans remettre en cause l'ensemble de l'application, tout en conservant une expérience cohérente pour les différents profils d'utilisateurs.

Zone du dépôt	Contenu principal	Responsabilité dans la solution
quizapp/	Application Spring Boot	Centralise la logique métier, la sécurité, les accès aux données,

		les endpoints REST et les services transverses.
quizapp_front/	Application Angular	Porte l'expérience utilisateur, les formulaires, les écrans métier et les flux de navigation.
z_notes/ et cartographies	Notes techniques et documents d'analyse	Capitalisent les comportements métiers, les correctifs apportés et les validations fonctionnelles.
docker-compose.yml	Services d'infrastructure locale	Décrit les dépendances techniques utiles au développement et aux validations locales.
z_mémoire/	Mémoire et supports académiques	Documente le projet, sa démarche et les résultats obtenus dans une logique d'ingénierie et de soutenance.

6.4 Implémentation des modules fonctionnels

L'implémentation fonctionnelle de SyneliaQuizApp ne s'est pas limitée à juxtaposer des écrans ou des endpoints isolés. Elle a consisté à faire converger plusieurs modules interdépendants autour d'un même objectif : rendre le processus de recrutement plus lisible, plus traçable et plus pilotable, depuis la publication d'une campagne jusqu'à la décision finale.

6.4.1 Gestion des campagnes et des formulaires

Le module des campagnes constitue le point d'entrée organisationnel du système. Il permet de créer une campagne, de la configurer, de lui associer un ou plusieurs formulaires et de contrôler les droits des collaborateurs qui peuvent la consulter, la modifier, suivre ses candidatures ou intervenir dans les étapes d'évaluation. Le projet ne se limite donc pas à une simple liste d'offres : la campagne devient une unité de pilotage métier qui structure les permissions, les parcours et les indicateurs.

La gestion des formulaires a été conçue avec une logique de réutilisation et de versionnement. Le rattachement entre campagne et formulaire repose sur une relation dédiée, ce qui permet de mutualiser un même formulaire entre plusieurs campagnes tout en gardant la possibilité de dupliquer ou d'isoler une version lorsqu'une divergence devient

nécessaire. Les stratégies d'affectation prises en charge, comme l'attribution aléatoire, séquentielle, conditionnée par un source token ou aléatoire sans répétition, montrent que le questionnaire technique n'est pas un bloc figé mais un sous-système adaptable au contexte de recrutement.

6.4.2 Gestion des candidatures et parcours public

Le module des candidatures couvre l'ensemble du flux candidat, depuis la soumission initiale jusqu'au suivi postérieur à l'envoi. Une candidature peut être déposée à partir d'un lien public, associée à une campagne donnée, puis enrichie par des pièces et des informations métier utiles au traitement du dossier. Un mécanisme de vérification d'email sécurise ensuite l'accès aux étapes sensibles, notamment au quiz technique.

Le projet intègre également un véritable parcours public de suivi. Chaque candidature reçoit un code de suivi permettant au candidat de consulter l'état d'avancement de son dossier, de reprendre un quiz encore actif, de répondre à une proposition d'entretien ou, dans certains cas, de retirer sa candidature. Cette continuité entre l'espace public et le back-office recruteur constitue un apport important de la solution, car elle réduit les ruptures d'information et améliore la transparence vis-à-vis du candidat.

6.4.3 Quiz technique et correction

Le quiz technique représente l'un des modules les plus structurants de l'application. Il a été conçu pour prendre en charge plusieurs types de questions, un chronométrage éventuel, la randomisation selon la configuration du formulaire et une gestion fine des tentatives. Le passage du quiz n'est plus rattaché à un simple jeton global mais à une occurrence explicite de tentative, ce qui améliore la traçabilité et prépare l'application à des scénarios plus riches de reprise ou de multi-occurrence.

La correction combine deux logiques complémentaires. D'un côté, les questions objectives sont notées automatiquement à partir des réponses soumises. De l'autre, les questions nécessitant une appréciation humaine peuvent passer par un circuit de correction manuelle, avec des statuts dédiés et des permissions précises pour les recruteurs habilités. Ce compromis entre automatisation et intervention humaine est essentiel pour préserver à la fois l'efficacité opérationnelle et la pertinence de l'évaluation technique.

6.4.4 Entretiens, évaluations et décisions

Le module des entretiens prolonge naturellement le quiz dans le pipeline de recrutement. Il permet de planifier une convocation, de notifier le candidat, de suivre sa réponse et de repositionner l'entretien si nécessaire. L'implémentation distingue clairement l'entretien planifié de la réponse candidat, ce qui rend possible une gestion plus réaliste des propositions, refus, reports et archivages.

Les évaluations associées aux entretiens vont au-delà d'une note globale. Elles peuvent porter des critères détaillés, des pondérations et des éléments de justification,

puis alimenter les agrégats utilisés dans les vues de suivi. La décision finale, qu'elle soit intermédiaire ou terminale, s'appuie ainsi sur un continuum cohérent : candidature, quiz, entretien, évaluation, décision. Cette chaîne métier renforce la lisibilité du dossier candidat pour les recruteurs et limite les pertes d'information entre étapes.

6.4.5 Notifications, permissions et pilotage

Les notifications et la gestion des permissions constituent la couche transverse qui rend l'application réellement exploitable en contexte collaboratif. Les droits ne sont pas gérés uniquement par rôle global, mais aussi par permissions contextualisées sur une campagne ou sur une entité donnée. Cette granularité permet d'ouvrir certaines actions à des recruteurs ou évaluateurs ciblés sans leur accorder un accès total à l'ensemble du système.

En parallèle, les notifications internes et les emails transactionnels assurent la circulation des informations critiques : nouvelles candidatures, convocations à entretien, retours sur demandes candidat, décisions ou alertes d'administration. Le tableau de bord unifie enfin l'accès à ces modules en proposant une lecture opérationnelle de l'activité, des listes métier, des filtres transverses et des écrans spécialisés par domaine.

6.5 Intégration et communication

La communication entre le frontend Angular et le backend Spring Boot repose sur une architecture API clairement séparée. Le frontend agit comme client de l'application et délègue l'essentiel des traitements métier au serveur via des appels HTTP centralisés dans des services applicatifs. En développement local, cette communication est simplifiée par les fichiers de proxy Angular, qui redirigent les requêtes vers l'API tout en évitant les frictions liées au CORS lors des itérations quotidiennes.

Les échanges de données se font principalement au format **JSON**, à travers des DTO qui stabilisent les contrats entre l'interface et les couches métier. Ce choix facilite la sérialisation, la validation côté client et la réutilisation des mêmes objets dans plusieurs écrans. Le projet suit par ailleurs une convention homogène d'API centrée sur des endpoints explicites, majoritairement exposés sous forme d'actions métier plutôt que comme simples opérations CRUD génériques. Cette approche se justifie par la richesse des cas d'usage : avancer une candidature, répondre à un entretien, publier une version de formulaire ou recalculer une évaluation ne relèvent pas tous d'une simple mise à jour standard.

Les flux de fichiers, en particulier autour des candidatures et des pièces associées, sont pris en charge par des points d'entrée dédiés. La solution sépare ainsi la donnée métier visible dans les écrans, comme le statut d'une candidature ou le lien de suivi public, des ressources binaires effectivement téléversées. Cette dissociation est importante car

elle permet de faire évoluer le stockage sans remettre en cause l'interface fonctionnelle exposée aux utilisateurs.

Sur le plan de la synchronisation, l'application repose moins sur un rafraîchissement brut de toutes les données que sur des contrats ciblés et des miroirs métier calculés côté backend. Par exemple, certaines informations de synthèse sur l'état courant d'une candidature, d'un entretien ou d'une évaluation sont recalculées puis renvoyées pour faciliter l'affichage dans les listes et tableaux de bord. Cette stratégie améliore la réactivité perçue du front tout en conservant un centre de vérité unique côté serveur. Elle est complétée par des mécanismes transverses comme les notifications, les WebSockets et les emails, qui assurent la diffusion rapide d'informations importantes aux différents acteurs du processus.

CHAPITRE 7

Tests et validation

Ce chapitre démontre la rigueur apportée à la vérification du système et à la validation des comportements attendus.

Positionnement méthodologique

Être honnête sur la profondeur réelle de couverture tout en valorisant les validations effectivement menées : tests métier ciblés, vérifications frontend, scénarios fonctionnels et contrôles manuels.

7.1 Stratégie de tests

La stratégie de validation adoptée dans le projet repose sur une combinaison pragmatique de vérifications automatiques et de contrôles manuels guidés par les parcours métier. L'objectif n'était pas de mettre en place une usine à tests exhaustive dès le départ, mais de sécuriser progressivement les zones les plus sensibles du système : permissions, contrats d'API, évolutions du modèle de données, parcours candidat et opérations de recrutement à forte valeur métier.

Cette stratégie s'organise selon plusieurs niveaux. Le premier niveau vérifie la stabilité technique du code par la compilation, les builds et les tests unitaires ciblés. Le deuxième niveau sécurise les interactions entre couches, notamment via des tests de contrôleurs, des validations de contrats JSON et des scénarios API rejoués avec Postman. Le troisième niveau correspond aux validations fonctionnelles menées sur l'interface et sur le parcours public, afin de s'assurer que les enchaînements réels restent cohérents du point de vue utilisateur.

Niveau	Support principal	Objectif de validation
Technique	Compilation, build, tests ciblés	Détecter rapidement une rupture de code, un contrat cassé ou une régression locale.

API	Tests backend et Postman	Vérifier les contrats HTTP, les règles métier et les réponses des endpoints critiques.
Interface	Specs Angular et vérifications UI	Contrôler les composants partagés, les écrans sensibles et les comportements attendus côté front.
Fonctionnel	Scénarios métier complets	Valider le parcours candidat, recruteur et administrateur dans des conditions proches de l'usage réel.

7.2 Tests unitaires

Le backend dispose d'un ensemble réel de tests unitaires et de tests de slice couvrant plusieurs zones importantes de l'application. On y trouve notamment des tests orientés métier sur les permissions, les formulaires, les questions, les entretiens, les évaluations, les tokens de source ou encore certains comportements de repository. Des tests de contrat viennent également sécuriser la sérialisation JSON de certains DTO et la cohérence d'endpoints exposés par les contrôleurs REST.

Cette couverture n'est pas uniforme sur toute l'application, mais elle se concentre sur les zones où une évolution peut facilement produire un effet de bord significatif. C'est particulièrement vrai pour les permissions contextualisées, la logique de versionnement des formulaires, les critères d'évaluation et les flux d'entretien. Ce ciblage traduit une approche réaliste : investir les tests là où le coût d'une régression serait élevé.

Côté frontend, plusieurs pages et composants disposent également de fichiers `.spec.ts`, en particulier autour des listes métier du dashboard, des modules de campagne, de décision, d'entretien, d'évaluation et des composants partagés de filtre. Ces tests restent plus modestes que ceux du backend, mais ils jouent un rôle utile pour verrouiller certains comportements d'interface, notamment lorsque des composants mutualisés sont réutilisés dans de nombreuses vues.

7.3 Tests d'intégration

Les validations d'intégration ont principalement porté sur les endpoints et sur la cohérence des échanges entre frontend et backend. Dans ce cadre, Postman a servi d'outil de référence pour rejouer rapidement des scénarios métier : création de campagne, progression de candidature, récupération d'un quiz public, réponse à un entretien, envoi

ou consultation de notifications, etc. Cette pratique est particulièrement adaptée à un projet dont plusieurs flux sont publics, tokenisés ou soumis à des permissions fines.

Les tests de contrôleurs présents côté backend jouent aussi un rôle d'intégration partielle, car ils vérifient la bonne réception des payloads HTTP, leur désérialisation et la propagation correcte vers la couche métier. Ils sont complétés, en développement courant, par des builds ciblés du backend et du frontend afin de sécuriser la compatibilité globale du dépôt après chaque série de corrections.

Enfin, l'intégration ne concerne pas seulement l'API. Elle inclut également les dépendances transverses comme la base MySQL, les migrations Flyway, les proxies frontend, la configuration SMTP active, ainsi que les services complémentaires comme Redis ou MinIO lorsqu'ils sont mobilisés par un scénario. Cette vision élargie permet de vérifier que l'application reste cohérente comme système et non comme simple somme de modules indépendants.

7.4 Tests fonctionnels

Les tests fonctionnels ont été organisés autour des parcours réels des trois grandes catégories d'utilisateurs : candidat, recruteur et administrateur. Pour le candidat, les scénarios essentiels couvrent la soumission d'une candidature, la vérification de l'adresse email, l'accès au quiz, la reprise éventuelle d'une tentative, la consultation du suivi public et la réponse à une proposition d'entretien. Ces scénarios permettent de contrôler la fluidité du parcours externe, qui constitue la façade la plus visible du système.

Pour le recruteur, les validations ont surtout porté sur la création et la configuration des campagnes, l'affectation des formulaires, la consultation des candidatures, la planification des entretiens, la correction éventuelle des réponses ouvertes, la saisie des évaluations et la prise de décision. La cohérence des permissions a fait l'objet d'une attention particulière, car une même page peut exposer des actions différentes selon les droits globaux ou contextuels de l'utilisateur.

Enfin, les tests administrateur ont concerné les données de référence, les rôles et permissions, la configuration applicative, les modèles d'email, les réglages SMTP et certaines vues techniques du dashboard. Ces validations sont importantes car elles conditionnent indirectement le bon fonctionnement des flux métier. Une application de recrutement peut paraître correcte côté utilisateur final tout en restant fragile si sa configuration transverse n'est pas fiable.

7.5 Tests de performance

Le projet n'a pas donné lieu à une campagne industrielle de tests de charge au sens strict. Il serait donc excessif de prétendre disposer de mesures complètes de montée en charge,

de latence sous forte concurrence ou de benchmarks comparatifs. En revanche, plusieurs choix d'architecture et plusieurs contrôles empiriques ont visé à préserver un niveau de performance cohérent avec le contexte d'usage attendu.

D'abord, la séparation frontend/backend permet de répartir proprement les responsabilités de calcul et de présentation. Ensuite, l'usage de DTO ciblés, de filtres de recherche, de vues de synthèse et d'agrégats calculés évite de surcharger inutilement l'interface avec des graphes de données trop volumineux. Enfin, l'existence de services auxiliaires dédiés, comme Redis ou MinIO, ouvre la voie à une meilleure spécialisation des charges lorsque le produit devra être davantage industrialisé.

Dans le contexte du mémoire, il est donc plus juste de parler d'une *performance raisonnablement maîtrisée* que d'une performance formellement benchmarkée. La priorité a été donnée à la justesse fonctionnelle, à la stabilité des parcours et à la qualité de structuration, tout en laissant ouverte la possibilité d'une optimisation plus poussée lors d'une phase ultérieure de mise en production élargie.

7.6 Tests de sécurité

La sécurité a été traitée à plusieurs niveaux. Le premier niveau concerne l'authentification et l'autorisation, assurées côté backend par Spring Security et prolongées par une logique métier fine de contrôle des permissions. L'application distingue en effet les droits globaux, liés au rôle système, des droits contextuels accordés sur une campagne ou une entité particulière. Cette granularité réduit le risque d'accès excessif et correspond mieux à la réalité collaborative d'un processus de recrutement.

Le deuxième niveau relève des validations d'entrée et de la protection des parcours publics. L'accès au quiz dépend notamment de la vérification de l'email, certaines actions publiques sont verrouillées en fonction de l'état de la candidature, et les liens de suivi s'appuient sur des codes dédiés plutôt que sur des identifiants exposés directement. Côté backend, plusieurs garde-fous évitent également de faire confiance à des informations transportées par le client lorsqu'une source authentifiée plus sûre est disponible, comme le principal JWT résolu côté serveur.

Enfin, la sécurité comprend aussi la protection des données sensibles et des services transverses. Les secrets SMTP restent traités comme des données d'écriture non relues en clair, les templates d'email échouent volontairement lorsqu'une variable obligatoire est absente, et les notifications d'administration signalent les défauts de configuration ou d'envoi au lieu de les masquer. Cette approche favorise une sécurité opérationnelle plus réaliste, centrée non seulement sur l'accès, mais aussi sur la fiabilité des traitements critiques.

7.7 Recettage et validation finale

La recette finale a suivi une logique incrémentale. Chaque évolution significative du projet a été confrontée à un petit ensemble de critères de validation : absence d'erreur de build, cohérence des contrats API, conformité du parcours utilisateur attendu, et non-régression sur les règles métier les plus sensibles. Cette méthode a permis de corriger rapidement les écarts au fil du développement, sans attendre une phase terminale trop lourde et trop risquée.

Les notes techniques, cartographies, guides Postman et check-lists produits pendant le projet ont joué un rôle important dans cette recette. Ils ont servi de mémoire opérationnelle pour rejouer des cas, comprendre l'origine d'un défaut et vérifier qu'un correctif n'en introduisait pas un autre ailleurs. Le recettage ne s'est donc pas limité à cocher des fonctionnalités présentes : il a consisté à confirmer que les comportements livrés restaient cohérents avec le besoin métier et avec les arbitrages pris en cours de projet.

Quatrième partie

Résultats et perspectives

Bilan du projet, valeur créée et axes d'amélioration futurs

CHAPITRE 8

Présentation des résultats

Ce chapitre a pour objectif de mettre en évidence les résultats effectivement obtenus à l'issue du projet, du point de vue fonctionnel, technique et méthodologique.

8.1 Démonstration de l'application

La démonstration de l'application peut être structurée autour de trois parcours complémentaires. Le premier est celui de l'administrateur, qui configure les référentiels, gère les droits, supervise les paramètres applicatifs et maintient les éléments transverses comme les modèles d'email ou la configuration SMTP. Ce parcours montre que la solution ne se limite pas à une interface métier, mais propose également les leviers d'administration nécessaires à une exploitation durable.

Le deuxième parcours est celui du recruteur. Il couvre la création d'une campagne, l'association d'un ou plusieurs formulaires, le partage d'accès avec d'autres intervenants, le suivi des candidatures, la planification des entretiens, la consultation des évaluations et la prise de décision. C'est dans ce parcours que la valeur métier de SyneliaQuizApp apparaît le plus clairement : l'information est regroupée, contextualisée et pilotable sans dépendre d'outils disparates.

Le troisième parcours est celui du candidat. Il comprend la soumission de la candidature, la vérification de l'adresse email, l'accès au quiz, le suivi public de l'état d'avancement et les interactions autour des entretiens. Ce parcours est particulièrement important dans une logique de qualité de service, car il traduit concrètement la manière dont l'entreprise communique avec ses candidats et structure leur expérience.

8.2 Fonctionnalités implémentées

Le tableau suivant résume les principales fonctionnalités effectivement prises en charge par la solution au terme du projet.

Module	Niveau de couverture	Synthèse
Campagnes et formulaires	Élevé	Création, configuration, partage d'accès, association de formulaires,

		gestion de versions et stratégies d'affectation.
Candidatures	Élevé	Soumission publique, vérification email, suivi candidat, retrait, consultation et traitement côté recruteur.
Quiz technique	Élevé	Tentatives, chronométrage, randomisation, reprise conditionnelle, notation automatique et correction manuelle partielle.
Entretiens	Élevé	Planification, réponse candidat, reprogrammation, archivage et suivi par occurrence.
Évaluations et décisions	Élevé	Critères détaillés, agrégats d'évaluation, décisions intermédiaires et finales avec notifications associées.
Administration transverse	Moyen à élevé	Permissions, configuration applicative, emails, SMTP, référentiels et vues techniques de supervision.

Cette couverture montre que le projet dépasse largement le stade d'un prototype d'interface. La plateforme prend en charge le cœur du processus de recrutement, avec une densité fonctionnelle suffisante pour soutenir un usage réel, même si certaines zones restent perfectibles ou méritent une industrialisation plus poussée.

8.3 Bilan technique

Sur le plan technique, le projet aboutit à une solution cohérente, fondée sur une séparation nette entre backend Spring Boot, frontend Angular et base relationnelle MySQL pilotée par migrations. Cette architecture présente un bon compromis entre robustesse, lisibilité et capacité d'évolution. Les principaux composants structurants du système sont identifiables : gestion centralisée des permissions, versionnement des formulaires, orchestration des parcours candidat, synchronisation des états métier et administration transverse du produit.

La qualité du code ne doit pas être évaluée uniquement à travers des indicateurs quantitatifs. Dans le cas présent, elle se manifeste surtout par la clarification progressive des responsabilités, la formalisation d'un nombre croissant de règles métier, l'introduction de tests ciblés sur les zones sensibles et la réduction de certains héritages techniques devenus ambiguës. Le projet conserve encore des marges d'amélioration, mais il dispose désormais d'une base technique suffisamment structurée pour accueillir de nouvelles itérations sans repartir de zéro.

Les performances observées dans le cadre de développement et de validation restent compatibles avec un usage interne raisonnable. Les parcours métier essentiels peuvent être exécutés localement de manière fluide, les builds sont reproductibles, et les dépendances d'infrastructure sont suffisamment explicites pour permettre une montée en qualité future. Ce constat reste volontairement prudent : il reflète un produit techniquement crédible, mais encore perfectible sur l'industrialisation avancée.

8.4 Retour d'expérience

Le projet a mis en évidence une réalité fréquente des applications métier : la complexité ne vient pas seulement du volume de code, mais surtout de la finesse des règles à articuler. Les difficultés majeures ont concerné la stabilité des parcours lorsqu'une même donnée devait être visible sous plusieurs angles, par exemple dans les listes, les écrans de détail, les flux publics et les agrégats de pilotage. Les notions de permission contextuelle, de versionnement des formulaires, d'occurrences d'entretien ou de tentatives de quiz ont exigé un travail d'alignement progressif entre base de données, backend et interface.

Parmi les décisions importantes prises pendant le projet figure la volonté de recentrer plusieurs comportements autour de sources de vérité plus explicites. Cela se traduit par l'usage accru des permissions d'entité, par la formalisation des tentatives de quiz, par la séparation entre formulaires partagés et versions publiées, ou encore par l'amélioration des contrats d'email et de notification. Ces choix ont parfois demandé davantage d'effort à court terme, mais ils renforcent la lisibilité et la solidité de la solution à moyen terme.

D'un point de vue méthodologique, le stage a également confirmé l'intérêt d'une documentation vivante : notes de debug, cartographies fonctionnelles, guides de test, backlog technique et synthèses de décision. Dans un projet riche en règles métier, cette capitalisation devient un levier de qualité à part entière, car elle facilite autant la correction des anomalies que la rédaction du mémoire lui-même.

CHAPITRE 9

Limites et perspectives

Ce dernier chapitre du corps du mémoire permet d'adopter un recul critique sur la solution réalisée et d'ouvrir vers des évolutions réalistes.

9.1 Limites du travail réalisé

Malgré les résultats obtenus, plusieurs limites demeurent. La première tient au niveau d'industrialisation des validations : le projet dispose de tests ciblés et de recettes sérieuses, mais pas encore d'une couverture automatisée exhaustive ni d'une chaîne de validation continue complète sur l'ensemble des parcours. Cette limite est classique dans un projet de stage, mais elle doit être reconnue car elle conditionne la vitesse et la sécurité des évolutions futures.

La deuxième limite concerne la maturité de certaines zones transverses. Les flux d'email, de notification, de configuration et certaines vues administratives ont fortement progressé, mais restent plus sensibles aux écarts de paramétrage que les parcours cœur. De même, l'application contient encore des mécanismes hérités ou des compatibilités temporaires qui ont été conservés pour accompagner la transition d'un modèle plus ancien vers une architecture plus propre.

Enfin, la solution n'a pas encore fait l'objet d'une mise à l'échelle complète dans un environnement de production exigeant. Les performances ont été jugées satisfaisantes dans le cadre du développement et des validations locales, mais la supervision, l'observabilité, le déploiement continu et les garanties de disponibilité devront être approfondis avant une généralisation à plus grande échelle.

9.2 Perspectives d'amélioration

Les perspectives d'amélioration sont nombreuses et crédibles. Sur le plan technique, une première priorité serait de renforcer la chaîne CI/CD, d'automatiser davantage les builds et d'étendre la couverture de tests sur les parcours les plus critiques, notamment ceux qui combinent permissions, états de candidature et interactions publiques. Une deuxième priorité porterait sur l'observabilité : journalisation métier enrichie, tableaux de bord de suivi, alerting sur les flux email ou sur les erreurs applicatives.

Sur le plan fonctionnel, l'application pourrait évoluer vers une meilleure exploitation analytique du recrutement, avec des indicateurs plus riches sur les taux de conversion, les délais de traitement, les performances des formulaires ou l'efficacité des campagnes. L'expérience utilisateur pourrait également être améliorée par une intégration plus poussée des calendriers, des interfaces de suivi encore plus pédagogiques pour les candidats et des outils de collaboration avancés pour les recruteurs.

Enfin, des évolutions structurantes pourraient concerner le déploiement et l'ouverture de la plateforme : séparation plus poussée des environnements, normalisation des procédures d'administration, documentation opérationnelle renforcée et préparation à une exploitation plus large au sein de l'entreprise.

9.3 Recommandations

Au regard des résultats obtenus, il est recommandé à SYNELIA de considérer Synelia-QuizApp comme une base applicative sérieuse à consolider, plutôt que comme un simple prototype ponctuel. Cette consolidation devrait passer par la stabilisation des règles métier les plus structurantes, la formalisation des priorités produit et la réduction progressive des mécanismes de compatibilité devenus moins utiles.

Il est également conseillé d'investir dans trois axes simultanés : la qualité logicielle, l'exploitation opérationnelle et l'expérience utilisateur. La qualité logicielle suppose davantage d'automatisation de test et de contrôle ; l'exploitation opérationnelle suppose des procédures claires pour la configuration, les déploiements et la supervision ; l'expérience utilisateur suppose d'améliorer encore la lisibilité des parcours pour les recruteurs comme pour les candidats.

Enfin, pour les futures évolutions, il conviendra de préserver le principe qui a le plus apporté de valeur au projet : faire reposer les choix techniques sur une lecture fine du besoin métier. C'est cette articulation entre rigueur logicielle et réalité du recrutement qui donnera à la solution sa pertinence durable.

Conclusion

La conclusion devra rappeler la problématique de départ, synthétiser la démarche suivie, mettre en valeur les résultats obtenus et souligner les contributions principales du projet au regard des besoins de SYNELIA.

Contenu obligatoire

- rappel de la problématique
- rappel de la démarche suivie
- résultats obtenus
- contributions principales
- limites du travail
- perspectives d'évolution

[Rédiger ici la conclusion finale du mémoire.]

Références bibliographiques

Version actuelle

La bibliographie sera complétée dans une seconde phase, une fois les ouvrages, articles et ressources en ligne fournis. Cette section est volontairement structurée dès maintenant pour faciliter l'injection ultérieure des références.

Ouvrages

[À compléter : ouvrages de génie logiciel, architecture logicielle, développement web, ingénierie des exigences, tests, sécurité.]

Documentation technique

[À compléter : Spring Boot, Angular, MySQL, MapStruct, Lombok, Typst, documentation REST.]

Articles et publications

[À compléter : articles scientifiques et publications professionnelles sur l'e-recrutement, les ATS, l'automatisation des évaluations techniques et l'ergonomie des plateformes numériques.]

Normes et standards

[À compléter : RFC, OWASP, OpenAPI, règles RGPD ou autres références normatives citées dans le mémoire.]

Annexes

Organisation recommandée

- Annexe A : captures d'écran de l'application
- Annexe B : diagrammes UML complémentaires
- Annexe C : scripts SQL significatifs
- Annexe D : extraits de code choisis
- Annexe E : documentation API
- Annexe F : guide d'installation et de déploiement
- Annexe G : manuel utilisateur
- Annexe H : planning du projet

Annexe A — Captures d'écran de l'application

[Insérer ici les captures principales par profil et par module.]

Annexe B — Diagrammes complémentaires

[Insérer ici les diagrammes UML ou d'architecture non inclus dans le corps principal.]

Annexe C — Scripts et artefacts techniques

[Insérer ici les éléments techniques complémentaires réellement utiles à la lecture.]

Table des matières

Introduction 1
1. Présentation générale du projet 5
    1.1. Contexte du projet 5
        1.1.1. L'entreprise SYNELIA et ses besoins 6
        1.1.2. Le processus de recrutement traditionnel 6
        1.1.3. Les défis et limitations actuels 6
        1.1.4. La nécessité d'une solution digitale 7
    1.2. Objectifs du projet 7
        1.2.1. Objectifs généraux 7
        1.2.2. Objectifs spécifiques 8
    1.3. Périmètre fonctionnel 8
        1.3.1. Modules couverts par l'application 8
        1.3.2. Acteurs du système 8
        1.3.3. Frontières du projet 9
    1.4. Contraintes du projet 9
2. Étude de l'existant et état de l'art 11
    2.1. Étude de l'existant 11
    2.2. Benchmark des solutions existantes 12
    2.3. État de l'art technologique 12
    2.4. Justification des choix technologiques 13
    2.5. Critique de l'existant et solution proposée 13
3. Analyse des besoins 16
    3.1. Méthodologie d'analyse 16
    3.2. Besoins fonctionnels 17
        3.2.1. Authentification et gestion des utilisateurs 17
        3.2.2. Gestion des données de référence 17
        3.2.3. Campagnes de recrutement 17
        3.2.4. Candidatures et suivi candidat 18
        3.2.5. Quiz technique 18
        3.2.6. Entretiens, évaluations et décisions 18
        3.2.7. Notifications et dashboard 19
    3.3. Besoins non fonctionnels 19
    3.4. Diagrammes des cas d'utilisation 20
    3.5. Règles de gestion 20

4. Conception de la solution 22
4.1. Architecture globale du système 22
4.2. Conception de la base de données 23
4.3. Diagrammes de classes 23
4.4. Diagrammes de séquence 24
4.5. Diagrammes d'activités et d'états 24
4.6. Maquettes et interfaces 25
4.7. Conception de l'API REST 25
5. Environnement technique et outils 28
5.1. Environnement de développement 28
5.2. Technologies backend 29
5.3. Technologies frontend 29
5.4. Base de données 30
5.5. Outils complémentaires 31
6. Implémentation de la solution 33
6.1. Organisation du code source 33
6.2. Implémentation du backend 33
6.3. Implémentation du frontend 34
6.4. Implémentation des modules fonctionnels 35
6.4.1. Gestion des campagnes et des formulaires 35
6.4.2. Gestion des candidatures et parcours public 36
6.4.3. Quiz technique et correction 36
6.4.4. Entretiens, évaluations et décisions 36
6.4.5. Notifications, permissions et pilotage 37
6.5. Intégration et communication 37
7. Tests et validation 39
7.1. Stratégie de tests 39
7.2. Tests unitaires 40
7.3. Tests d'intégration 40
7.4. Tests fonctionnels 41
7.5. Tests de performance 41
7.6. Tests de sécurité 42
7.7. Recettage et validation finale 43
8. Présentation des résultats 45
8.1. Démonstration de l'application 45
8.2. Fonctionnalités implémentées 45
8.3. Bilan technique 46
8.4. Retour d'expérience 47

9. Limites et perspectives 48
9.1. Limites du travail réalisé 48
9.2. Perspectives d'amélioration 48
9.3. Recommandations 49
Conclusion 50
Références bibliographiques 51
Annexes 52
Annexe A — Captures d'écran de l'application 52
Annexe B — Diagrammes complémentaires 52
Annexe C — Scripts et artefacts techniques 52